



HAWASSA UNIVERSITY

Department of Information System

ADVANCED DATABASE SYSTEMS

Year III - Section 1 Group 4

PROJECT TITLE: SUPERMARKET MANAGEMENT SYSTEM

No.	Group Member	ID No.
1	Dagim Huruma	1726/16
2	Abenezer Abera	1072/16
3	Lensa Habtamu	2675/16
4	Abubeker Dilgeba	1159/16
5	Netsanet Mulugeta	0650/15
6	Biniyam Eyob	1608/16
7	Nahom Taye	3109/16

Submitted to: Mr. Daniel T.

Submission Date: Jan 01, 2026G.C

Table of Contents

1. Introduction.....	1
2. Problem Statement.....	1
3. Project Objectives.....	1
4. System Overview.....	2
5. Database Design.....	2
5.1 Conceptual Design.....	2
5.2 Logical Design.....	3
6. Implementation Details.....	4
6.1 Database Setup (Code Snippet).....	4
6.2 Table Creation.....	4
6.3 Sample Data Insertion.....	4
7. Transaction Management and Concurrency Control.....	5
7.1 Dirty Read Demonstration.....	5
7.2 Lost Update Demonstraion.....	6
7.3 Deadlock Demonstration.....	8
8. Query Processing and Optimization.....	8
9. Security and Access Control.....	12
10. Auditing.....	13
11. Backup and Recovery Strategy.....	13
12. Testing and Evaluation.....	14
13. Challenges and Lessons Learned.....	15
14. Conclusion.....	15
15. References.....	15

1. Introduction

The Supermarket Management System is an advanced database project designed to demonstrate the practical application of core and advanced database concepts. The system supports essential supermarket operations such as product management, billing, payments, employee handling, supplier coordination, and inventory control. In addition to basic database design, the project integrates advanced database features including transaction management, concurrency control, query optimization, security mechanisms, auditing, and backup strategies.

This project builds upon foundational database principles and extends them to address real-world challenges such as concurrent access, data consistency, and recovery from failures, and controlled access to sensitive data.

2. Problem Statement

Modern supermarkets handle a large number of simultaneous transactions involving sales, inventory updates, and employee operations. Without proper transaction management and concurrency control, issues such as dirty reads, lost updates, and deadlocks may occur. Furthermore, lack of security and recovery mechanisms can lead to data breaches or data loss.

The problem addressed in this project is the design and implementation of a robust database system that ensures data consistency, security, reliability, and performance under concurrent usage.

3. Project Objectives

The main objectives of this project are:

- ✓ To design a relational database schema for a supermarket system
- ✓ To implement transaction management and concurrency control mechanisms
- ✓ To demonstrate common concurrency anomalies and their handling

- ✓ To apply query optimization techniques for performance improvement
- ✓ To enforce security using role-based access control
- ✓ To implement auditing and backup strategies for data integrity and recovery

4. System Overview

The system manages the following core entities:

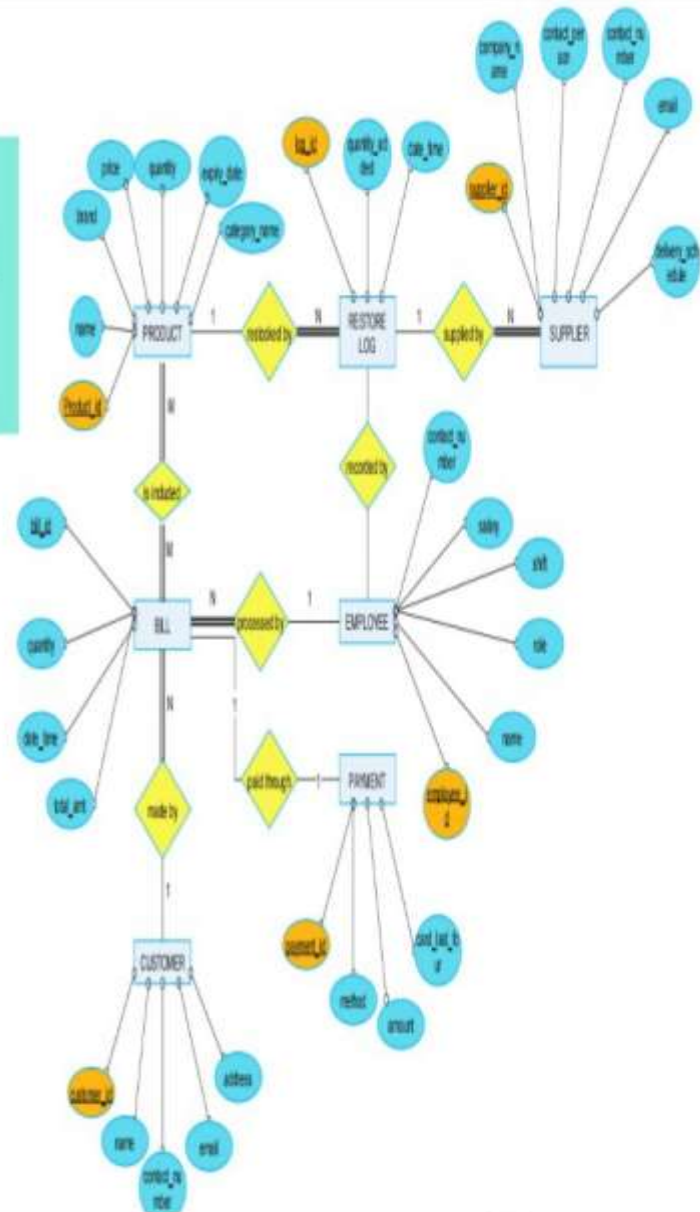
- Customer
- Product
- Employee
- Supplier
- Bill
- Payment
- Restock_Log

These entities interact to support sales transactions, inventory updates, restocking activities, and reporting.

5. Database Design

5.1 Conceptual Design

The conceptual design is represented using an Entity-Relationship (ER) model. The ER diagram captures entities, their attributes, and relationships reflecting real-world supermarket operations.



5.2 Logical Design

The logical design transforms the ER model into relational tables with primary keys and foreign key constraints. Normalization up to Third Normal Form (3NF) was applied to eliminate redundancy and maintain data integrity.

6. Implementation Details

6.1 Database Setup (Code Snippet)

```
1      /* =====
2      Supermarket Management System (Improved)
3      Phase 2: Implementation (Advanced Database Systems)
4      RDBMS: SQL Server
5      ===== */
6
7      -----
8      -- 1. DATABASE SETUP
9      IF EXISTS (SELECT name FROM sys.databases WHERE name = 'Supermarkeet')
10     BEGIN
11         ALTER DATABASE Supermarkeet SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
12         DROP DATABASE Supermarkeet;
13     END
14     GO
15
16     CREATE DATABASE supermarkeet;
17     GO
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2025-12-31T05:47:31.9560544-03:00

The database was implemented using Microsoft SQL Server. Initial configuration included setting the recovery model, enabling statistics updates, and disabling auto-shrink for performance and reliability.

6.2 Table Creation

All tables were created with appropriate data types, primary keys, foreign keys, and constraints such as CHECK and NOT NULL to enforce domain and entity integrity.

6.3 Sample Data Insertion

-- CUSTOMER

INSERT INTO dbo.Customer (name, contact_number, email, address)

VALUES ('Alice Johnson', '0912345678', 'alice@example.com', 'Addis Ababa');

-- EMPLOYEE

INSERT INTO dbo.Employee (name, role, shift, salary, contact_number)

```
VALUES ('Daniel Mekonnen', 'Cashier', 'Morning', 5000, '0934567890');
```

```
-- PRODUCT
```

```
INSERT INTO dbo.Product (name, brand, price, quantity_in_stock, expiry_date,  
category_name)
```

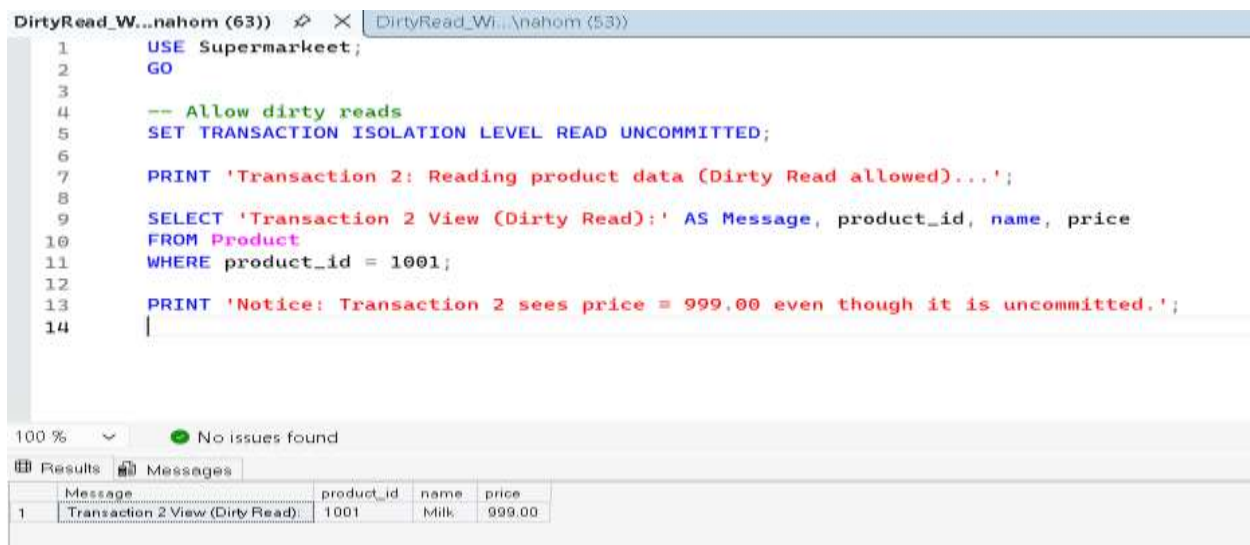
```
VALUES ('Milk', 'DairyPure', 45, 50, '2025-01-10', 'Dairy');
```

Sample records were inserted into each table to simulate real-world supermarket operations and support testing and demonstrations.

7. Transaction Management and Concurrency Control

7.1 Dirty Read Demonstration

Before dirty read



```
1 USE Supermarkeet;  
2 GO  
3  
4 -- Allow dirty reads  
5 SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;  
6  
7 PRINT 'Transaction 2: Reading product data (Dirty Read allowed)...';  
8  
9 SELECT 'Transaction 2 View (Dirty Read):' AS Message, product_id, name, price  
10 FROM Product  
11 WHERE product_id = 1001;  
12  
13 PRINT 'Notice: Transaction 2 sees price = 999.00 even though it is uncommitted.';  
14
```

100 % No issues found

Message	product_id	name	price
1 Transaction 2 View (Dirty Read):	1001	Milk	999.00

After Dirty Read

```
DIRTY ROLL...nahom (82)) X DirtyRead_Wi...nahom (63)) DirtyRead_Wi...nahom (53))
1  -- Back in Window 2
2  ROLLBACK TRANSACTION;
3
4  BEGIN
5      SELECT 'Final Price (After Rollback):' AS Message, product_id, name, price
6      FROM Product
7      WHERE product_id = 1001;
8
9      PRINT 'Transaction 1 rolled back. Price is restored to 45.00.';
10 END;
11 GO
12
```

100 % No issues found

Results Messages

	Message	product_id	name	price
1	Final Price (After Rollback):	1001	Milk	999.00

This snippet shows how a transaction can read uncommitted data from another transaction, leading to a dirty read anomaly.

A dirty read scenario was demonstrated using two concurrent transactions, where one transaction read uncommitted data from another transaction that was later rolled back.

7.2 Lost Update Demonstraion

Transaction 1


```
2 GO
3
4 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- Prevent lost updates
5 BEGIN TRANSACTION;
6
7 PRINT 'Transaction 1: Reading product stock...';
8
9 SELECT quantity_in_stock
10 FROM Product
11 WHERE product_id = 1001;
12
13 PRINT 'Transaction 1: Simulating delay...';
14 WAITFOR DELAY '00:00:10'; -- Simulate processing time
15
16 PRINT 'Transaction 1: Selling 1 item...';
17
18 UPDATE Product
19 SET quantity_in_stock = quantity_in_stock - 1
20 WHERE product_id = 1001;
21
22 COMMIT TRANSACTION;
23
24 PRINT 'Transaction 1 committed.';
25 GO
26
```

100 % No issues found

Results Messages

quantity_in_stock
37

Transaction 2

```
1 USE Supermarkeet;
2 GO
3
4 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- Prevent lost updates
5 BEGIN TRANSACTION;
6
7 PRINT 'Transaction 2: Reading product stock...';
8
9 SELECT quantity_in_stock
10 FROM Product
11 WHERE product_id = 1001;
12
13 PRINT 'Transaction 2: Selling 1 item...';
14
15 UPDATE Product
16 SET quantity_in_stock = quantity_in_stock - 1
17 WHERE product_id = 1001;
18
19 COMMIT TRANSACTION;
20
21 PRINT 'Transaction 2 committed.';
22 GO
23
```

100 % No issues found

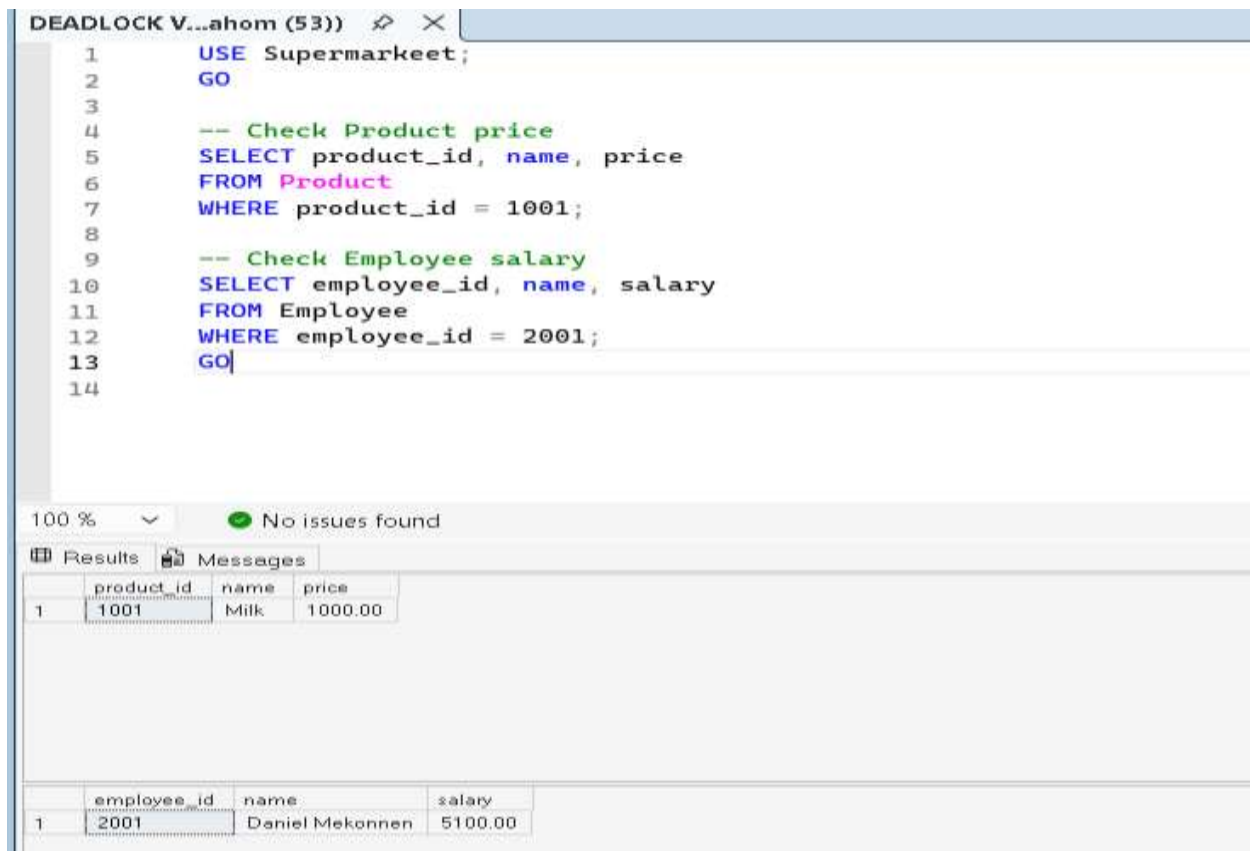
Results Messages

quantity_in_stock
36

When two transactions execute this logic concurrently without proper locking, one update may overwrite the other, resulting in a lost update.

The lost update problem was demonstrated by allowing two transactions to read the same stock value and update it concurrently, resulting in one update being overwritten.

7.3 Deadlock Demonstration



```
1  USE Supermarkeet;
2  GO
3
4  -- Check Product price
5  SELECT product_id, name, price
6  FROM Product
7  WHERE product_id = 1001;
8
9  -- Check Employee salary
10 SELECT employee_id, name, salary
11 FROM Employee
12 WHERE employee_id = 2001;
13 GO
14
```

100 % No issues found

Results Messages

	product_id	name	price
1	1001	Milk	1000.00

	employee_id	name	salary
1	2001	Daniel Mekonnen	5100.00

These opposing lock requests create a circular wait condition, resulting in a deadlock.

A deadlock scenario was created using two transactions accessing the Product and Employee tables in opposite order. SQL Server automatically detected the deadlock and resolved it by rolling back one transaction.

8. Query Processing and Optimization

Complex analytical queries were implemented to generate sales reports and product statistics. Indexes were created on frequently accessed columns to improve query performance. Execution plans were analyzed before and after indexing to demonstrate optimization benefits.

Demonstration of 5 sql Query optimization

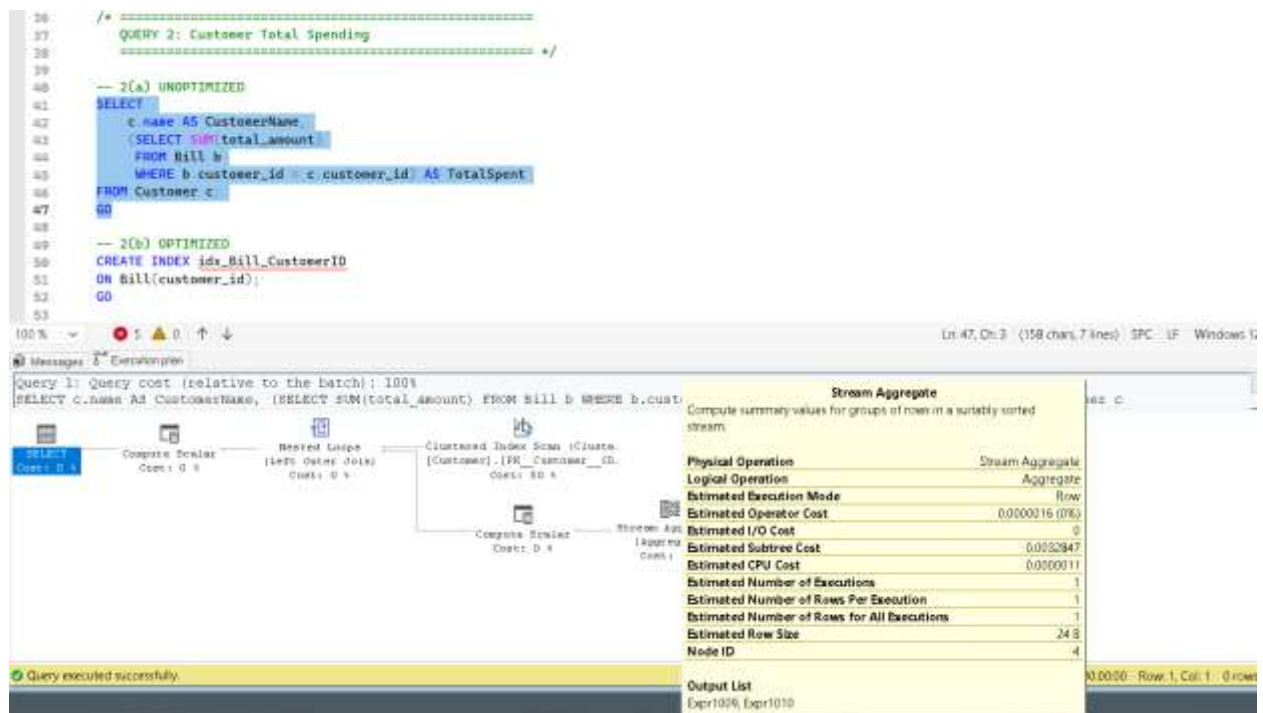
```
1      /* =====
2      ADVANCED QUERY OPTIMIZATION
3      Supermarket Management System
4      ===== */
5
6      USE Supermarkeet;
7      GO
8
9      /* =====
10     QUERY 1: Total Revenue per Product
11     ===== */
12
13     -- 1(a) UNOPTIMIZED
14     SELECT
15         p.name AS ProductName,
16         (SELECT SUM(bp.quantity * p.price)
17          FROM Bill_Product bp
18          WHERE bp.product_id = p.product_id) AS TotalRevenue
19     FROM Product p;
20     GO
21
22     -- 1(b) OPTIMIZED
23     CREATE INDEX idx_BillProduct_ProductID
24     ON Bill_Product(product_id);
25     GO
26
27     SELECT
28         p.name AS ProductName,
29         SUM(bp.quantity * p.price) AS TotalRevenue
30     FROM Product p
31     JOIN Bill_Product bp
32     ON p.product_id = bp.product_id
33     GROUP BY p.name;
34     GO
35
36     /* =====
37     QUERY 2: Customer Total Spending
38     ===== */
39
40     -- 2(a) UNOPTIMIZED
41     SELECT
42         c.name AS CustomerName,
43         (SELECT SUM(total_amount)
44          FROM Bill b
45          WHERE b.customer_id = c.customer_id) AS TotalSpent
46     FROM Customer c;
47     GO
48
49     -- 2(b) OPTIMIZED
50     CREATE INDEX idx_Bill_CustomerID
51     ON Bill(customer_id);
52     GO
53
54     SELECT
55         c.name AS CustomerName,
56         SUM(b.total_amount) AS TotalSpent
57     FROM Customer c
58     JOIN Bill b
59     ON c.customer_id = b.customer_id
60     GROUP BY c.name;
61     GO
62
63     /* =====
64     QUERY 3: Products Near Expiry
65     ===== */
66
67     -- 3(a) UNOPTIMIZED
68     SELECT name AS ProductName, expiry_date
69     FROM Product
70     WHERE DATEDIFF(DAY, GETDATE(), expiry_date) <= 30;
71     GO
72
73     -- 3(b) OPTIMIZED
74     CREATE INDEX idx_Product_ExpiryDate
75     ON Product(expiry_date);
76     GO
```

```

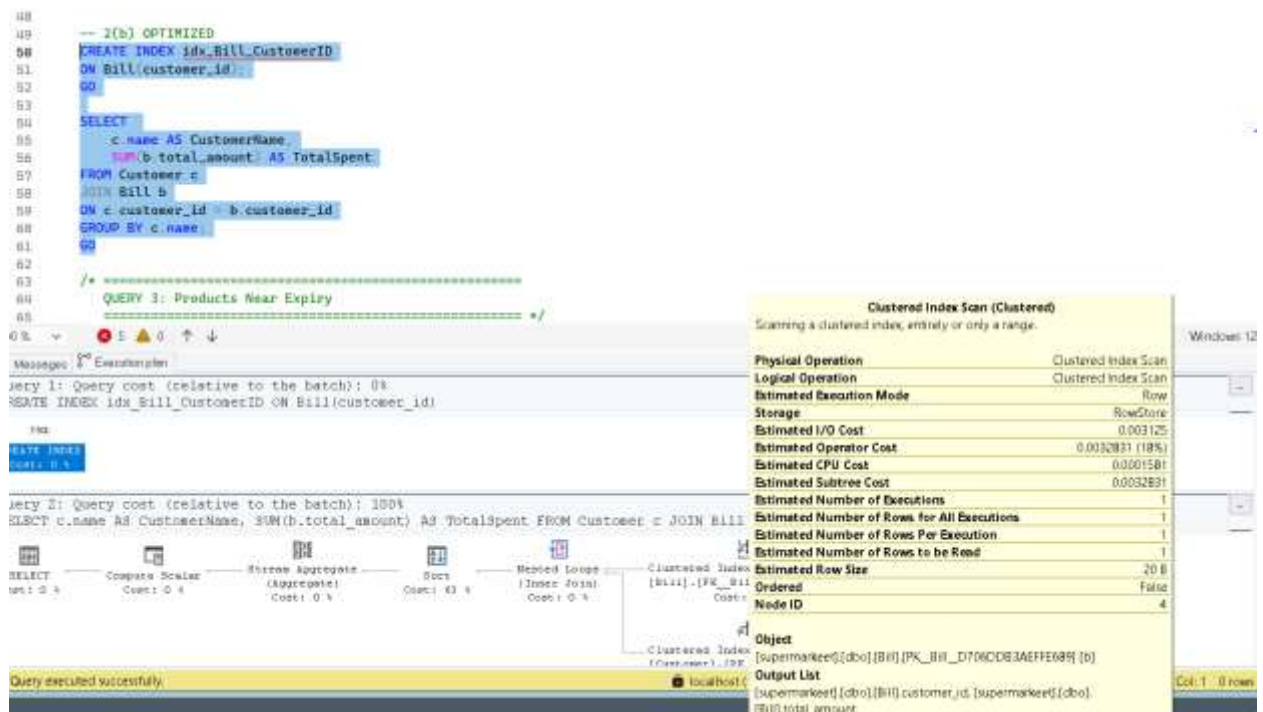
79 FROM Product
80 WHERE expiry_date <= DATEADD(DAY, 30, GETDATE());
81 GO
82
83 /* =====
84 QUERY 4: Employee Sales Count
85 ===== */
86
87 -- 4(a) UNOPTIMIZED
88 SELECT
89     e.name AS EmployeeName,
90     (SELECT COUNT(*)
91      FROM Bill b
92      WHERE b.employee_id = e.employee_id) AS NumberOfSales
93 FROM Employee e;
94 GO
95
96 -- 4(b) OPTIMIZED
97 CREATE INDEX idx_Bill_EmployeeID
98 ON Bill(employee_id);
99 GO
100
101 SELECT
102     e.name AS EmployeeName,
103     COUNT(b.bill_id) AS NumberOfSales
104 FROM Employee e
105 LEFT JOIN Bill b
106 ON e.employee_id = b.employee_id
107 GROUP BY e.name;
108 GO
109
110 /* =====
111 QUERY 5: Low Stock Products That Have Been Sold
112 ===== */
113
114 -- 5(a) UNOPTIMIZED
115 SELECT DISTINCT name AS ProductName
116 FROM Product
117 WHERE quantity_in_stock < 50
118 AND product_id IN (
119     SELECT product_id
120     FROM Bill_Product
121 );
122 GO
123
124 -- 5(b) OPTIMIZED
125 CREATE INDEX idx_Product_Quantity
126 ON Product(quantity_in_stock);
127 GO
128
129 SELECT DISTINCT p.name AS ProductName
130 FROM Product p
131 JOIN Bill_Product bp
132 ON p.product_id = bp.product_id
133 WHERE p.quantity_in_stock < 50;
134 GO
135

```

Result sample 1 of unoptimized query



Result sample 1 of optimized query



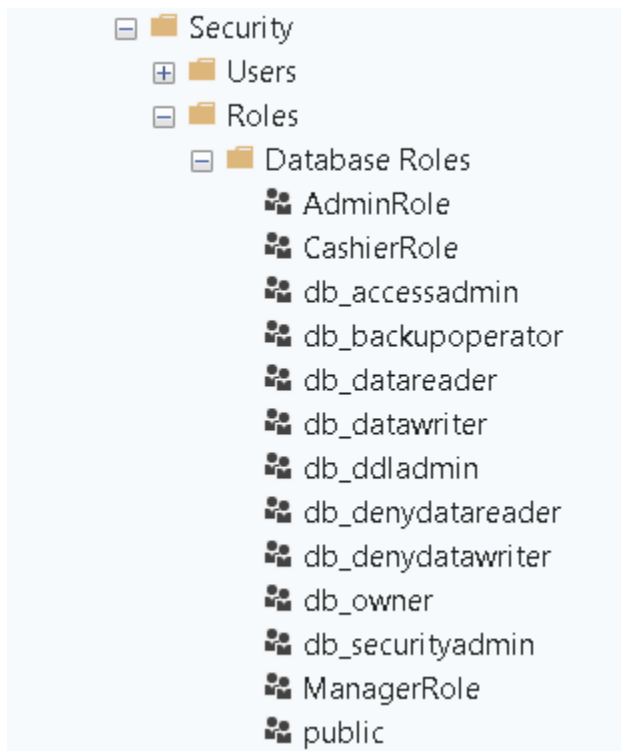
9. Security and Access Control

Role-Based Access Control ensures that users only perform operations allowed by their responsibilities.

Role-Based Access Control (RBAC) was implemented to restrict database access:

- ✓ **Cashier role:** Limited access to billing and product viewing
- ✓ **Manager role:** Full access to product and employee management
- ✓ **Admin role:** Full system-level access, including managing users/roles, security settings, and backups/recovery

This ensures data confidentiality and controlled access based on user responsibilities.



10. Auditing

An audit mechanism was implemented using triggers to track changes made to product prices. The audit table records old and new values along with timestamps, supporting accountability and traceability.

Results	Execution plan	Client Statistics			
		Trial 2	Trial 1	Average	
Client Execution Time		06:06:11	06:05:27		
Query Profile Statistics					
Number of INSERT, DELETE and UPDATE statements	0	→	0	→	0.0000
Rows affected by INSERT, DELETE, or UPDATE statements	0	→	0	→	0.0000
Number of SELECT statements	2	↑	0	→	1.0000
Rows returned by SELECT statements	2	↑	0	→	1.0000
Number of transactions	0	→	0	→	0.0000
Network Statistics					
Number of server roundtrips	4	→	4	→	4.0000
TDS packets sent from client	4	→	4	→	4.0000
TDS packets received from server	4	→	4	→	4.0000
Bytes sent from client	1776	↓	1890	→	1833.0000
Bytes received from server	3712	↑	543	→	2127.5000
Time Statistics					
Client processing time	7	↓	9	→	8.0000
Total execution time	11	↓	15	→	13.0000
Wait time on server replies	4	↓	6	→	5.0000

It demonstrates performance client statistics for your queries:

How many SELECT/UPDATE statements ran, how many rows they returned, how many network round-trips and bytes were sent/received, and how long the query took (processing time, total execution time, wait time).

11. Backup and Recovery Strategy

This backup strategy enables full database recovery in case of system failure.

A full database backup strategy was implemented to protect against data loss. The database recovery model supports transaction log backups and point-in-time recovery.


```
Messages
Processed 736 pages for database 'supermarkeet', file 'supermarkeet' on file 1.
Processed 2 pages for database 'supermarkeet', file 'supermarkeet_log' on file 1.
BACKUP DATABASE successfully processed 738 pages in 0.025 seconds (230.468 MB/sec).
Processed 112 pages for database 'supermarkeet', file 'supermarkeet' on file 1.
Processed 2 pages for database 'supermarkeet', file 'supermarkeet_log' on file 1.
BACKUP DATABASE WITH DIFFERENTIAL successfully processed 114 pages in 0.021 seconds (42.224 MB/sec).
Processed 8 pages for database 'supermarkeet', file 'supermarkeet_log' on file 1.
BACKUP LOG successfully processed 8 pages in 0.004 seconds (15.625 MB/sec).

Completion time: 2025-12-31T06:08:57.3487939-03:00
```

It shows the backup/auditing output from SQL Server:

- Confirms that the full, differential, and log backups of supermarkeet ran successfully.
- Reports how many data pages were processed, how long each backup took, and the throughput (MB/sec).

12. Testing and Evaluation

The system was tested for:

- ✓ Data integrity enforcement
- ✓ Correct execution of transactions
- ✓ Handling of concurrency anomalies
- ✓ Security role enforcement
- ✓ Successful backup creation

All tests confirmed that the database behaves as expected under normal and concurrent operations.

13. Challenges and Lessons Learned

Key challenges included designing reliable concurrency demonstrations and ensuring deadlocks occurred consistently. The project reinforced the importance of transaction isolation, proper locking strategies, and systematic database design.

14. Conclusion

This project successfully demonstrates the design and implementation of an advanced database system for supermarket management. By integrating transaction management, concurrency control, security, auditing, and backup strategies, the system meets the requirements of an Advanced Database Systems course and reflects real-world database challenges.

15. References

- Microsoft SQL Server Documentation
- Silberschatz, Korth, Sudarshan – *Database System Concepts*
- W3Schools SQL Tutorial